Examples of Sequence-to-Sequence Tasks

Beyond translation, this method is used in various fields, such as:

1. Style Transfer

Transforming one writing style into another.
Examples:
Formal English → Casual English
Shakespearean English → Modern English

2. Generative Question Answering

Generating answers to questions based on a given context.

Ways to Build a New Translation Model

If sufficient datasets are available in two or more languages, a translation model can be trained from scratch. However, this is time-consuming. Therefore, a pretrained model is usually fine-tuned, which is faster and more efficient.
Some popular pretrained models are:
mT5
mBART
MarianMT
These models can be improved by retraining them for specific language pairs or on specific datasets.

Using the Marian Model

In this section, we will use the pretrained MarianMT model for English-to-French translation. It is a popular model from Hugging Face.
Model Used:
Helsinki-NLP/opus-mt-en-fr
This model is trained on a large amount of English and French text from the OPUS dataset.

KDE4 Dataset

In this chapter, we will use the KDE4 dataset. It is built from localized files of KDE software, where technical terms have been accurately translated.
Code to Load the Dataset:

```
from datasets import load_dataset
raw_datasets = load_dataset("kde4", lang1="en", lang2="fr")
```

This dataset contains a total of 210,173 sentence pairs.

Train and Validation Data Split

Since the dataset has only one split, it must be divided into training and validation sets.

```
split_datasets = raw_datasets["train"].train_test_split(
train_size=0.9, seed=20
)
split_datasets["validation"] = split_datasets.pop("test")
```

After splitting:
Training Data: 189,155 sentence pairs
Validation Data: 21,018 sentence pairs

Example from the Dataset

```
split_datasets["train"][1]["translation"]
```

Output:

```
{
'en': 'Default to expanded threads',
'fr': 'Par défaut, développer les fils de discussion'
}
```

Here, the English sentence has been accurately translated into French.

Translation Using the Pretrained Model

```
from transformers import pipeline
model_checkpoint = "Helsinki-NLP/opus-mt-en-fr"
translator = pipeline("translation", model=model_checkpoint)
translator("Default to expanded threads")
```

Output:

```
[{'translation_text': 'Par défaut pour les threads élargis'}]
```

Here, it can be observed that the model keeps the word “threads” in English instead of translating it.

Another Example: “Plugin”

In the KDE4 dataset, the word “plugin” has been formally translated.
Dataset Translation:

```
{
'en': 'Unable to import %1 using the OFX importer plugin.',
'fr': "Impossible d'importer %1 en utilisant le module d'extension d'importation OFX."
}
```

However, the pretrained model translates it as:
"Impossible d'importer %1 en utilisant le plugin d'importateur OFX."
Here, the word “plugin” is retained in English.

After fine-tuning, the model can learn to produce more accurate translations according to the dataset style.

Exercise

The word “Email” is often used in English even in the French language.
Task:
Find the first example containing the word “email” in the KDE4 training dataset.
Observe how it has been translated.
Translate the same sentence using the pretrained model and compare the results.

Data Processing (Processing the Data)

Now we will prepare the data for training the translation model. A machine learning model cannot directly understand text, so the text must first be converted into numerical form. This process is called Tokenization. In tokenization, each word or subword is converted into a specific token ID.
For this translation task, both the input (English) and the target (French) languages must be tokenized.

Creating the Tokenizer

We will use the tokenizer of the MarianMT pretrained model for English-to-French translation.

```
from transformers import AutoTokenizer
model_checkpoint = "Helsinki-NLP/opus-mt-en-fr"
tokenizer = AutoTokenizer.from_pretrained(
model_checkpoint, return_tensors="pt"
)
```

Here, AutoTokenizer automatically loads the appropriate tokenizer. The argument return_tensors="pt" creates PyTorch tensors. You can also load a tokenizer from another model on the Hugging Face Hub or from a local folder. The Helsinki-NLP organization provides models for more than a thousand languages.

Instructions for Multilingual Tokenizers

If multilingual models such as mBART, mBART-50, or M2M100 are used, the source and target language codes must be specified.

```
tokenizer.src_lang = "en_XX"
tokenizer.tgt_lang = "fr_XX"
```

Tokenizing the Input and Target

For a translation model, both the input and the target must be tokenized properly. For this purpose, the text_target argument is used.

```
en_sentence = split_datasets["train"][1]["translation"]["en"]
fr_sentence = split_datasets["train"][1]["translation"]["fr"]
inputs = tokenizer(en_sentence, text_target=fr_sentence)
print(inputs)
```
The output will look like this:
```
{
'input_ids': [...],
'attention_mask': [...],
'labels': [...]
}
```
Here, input_ids represent the token IDs of the English sentence, attention_mask indicates the important tokens, and labels represent the token IDs of the French sentence.

Example of Incorrect Tokenization

If the target language is not specified correctly, incorrect tokens are generated.
```
wrong_targets = tokenizer(fr_sentence)
print(tokenizer.convert_ids_to_tokens(wrong_targets["input_ids"]))
print(tokenizer.convert_ids_to_tokens(inputs["labels"]))
```
In this case, it can be observed that processing French text with an English tokenizer generates unnecessarily more tokens because the tokenizer does not properly recognize French words.

Creating the Preprocessing Function

A function is created to apply tokenization to the entire dataset.
```
max_length = 128
def preprocess_function(examples):
inputs = [ex["en"] for ex in examples["translation"]]
targets = [ex["fr"] for ex in examples["translation"]]
model_inputs = tokenizer(
inputs,
text_target=targets,
max_length=max_length,
truncation=True
)
return model_inputs
```
Here, max_length = 128 defines the maximum length of the input and output sequences, and truncation=True removes excessively long sentences.

Special Instructions for the T5 Model

If the T5 model is used, a prefix must be added at the beginning of the input.
Example:
translate English to French:

Instructions Regarding the Padding Token

During model training, the value of the padding token is set to -100 so that it is ignored during loss computation. This task is usually performed automatically by the Data Collator.

Applying Preprocessing to the Entire Dataset

Now we will apply the preprocessing function to the complete dataset.
tokenized_datasets = split_datasets.map(
preprocess_function,
batched=True,
remove_columns=split_datasets["train"].column_names,
)
Here, batched=True speeds up processing by handling data in batches, and remove_columns removes unnecessary columns.

Fine-Tuning the Model Using the Trainer API

Now we will fine-tune the translation model. For this task, the Hugging Face Trainer API is used. However, instead of the standard Trainer, Seq2SeqTrainer will be used because it is specifically designed for sequence-to-sequence tasks.
This Trainer can generate outputs from inputs using the generate() method during evaluation, which is extremely important for translation tasks.

Loading the Model

First, we need to load a pretrained model. Here, AutoModelForSeq2SeqLM will be used.
from transformers import AutoModelForSeq2SeqLM
model = AutoModelForSeq2SeqLM.from_pretrained(model_checkpoint)
Explanation:
AutoModelForSeq2SeqLM is used for sequence-to-sequence language models.
This model has already been trained for translation tasks, so no warnings about missing weights appear.
It can perform translations directly and can be further improved through fine-tuning.

Data Collation

During model training, data is organized into batches. However, since sentence lengths are not equal, padding is required. This task is handled by the Data Collator.
The standard DataCollatorWithPadding cannot be used here because it only pads the inputs. In translation models, the labels must also be padded. Therefore, we use DataCollatorForSeq2Seq.
from transformers import DataCollatorForSeq2Seq
data_collator = DataCollatorForSeq2Seq(

tokenizer=tokenizer,
model=model
)
Why It Is Used:
It pads both inputs and labels.
It sets the padding value of labels to -100 so that they are ignored during loss computation.
It automatically creates decoder input IDs.

Testing the Data Collator

The following code tests the Data Collator with a few samples.
batch = data_collator(
[tokenized_datasets["train"][i] for i in range(1, 3)]
)
batch.keys()
Output:
dict_keys(['attention_mask', 'input_ids', 'labels', 'decoder_input_ids'])
Here, four components are present:
input_ids – Input tokens
attention_mask – Indicates important tokens
labels – Target tokens
decoder_input_ids – Decoder inputs

Verifying Label Padding

batch["labels"]
Here, shorter sentences are padded with -100. This value is ignored during loss computation.

Verifying Decoder Input IDs

batch["decoder_input_ids"]
These are shifted versions of the labels. That is, the decoder predicts the next word based on the previous word.

Viewing the Original Labels

for i in range(1, 3):
print(tokenized_datasets["train"][i]["labels"])
Here, the original token IDs are displayed, which the Data Collator prepares for training through padding and shifting.

Important Concepts

1. Seq2SeqTrainer

Specifically designed for sequence-to-sequence tasks.
Uses generate() during evaluation to produce translations.

2. Dynamic Padding

Padding is applied based on the maximum sequence length within each batch.
This saves memory and computational power.

3. Padding Value -100

Used to ignore padding tokens during loss computation.

4. Decoder Input IDs

Shifted versions of the labels.
Helps generate accurate translations.

Now our model, data, and Data Collator are ready. In the next step, we will define evaluation metrics such as the BLEU score.

Metrics

Selecting the correct metric is extremely important for evaluating the performance of a translation model. A key advantage of Seq2SeqTrainer is that it uses the generate() method during evaluation, which helps assess real-world performance.

Seq2SeqTrainer and the generate() Method

Seq2SeqTrainer is an advanced version of the standard Trainer. It uses the generate() method during evaluation and prediction.
During Training:
The model uses decoder_input_ids to make faster predictions.
During Inference or Evaluation:
Since labels are not available, the model generates tokens one by one. This process is completed through the generate() method.
To enable this feature, predict_with_generate=True must be set.

BLEU Score

The most widely used metric for evaluating translation models is BLEU (Bilingual Evaluation Understudy). It was introduced in 2002 by Kishore Papineni and his colleagues.
What the BLEU Score Measures:
How accurate the model's translation is.
How closely the generated sentence matches the target sentence.
It penalizes repetitive words and excessively short translations.

However, BLEU does not directly measure grammatical correctness or semantic depth; it relies on statistical comparison.

SacreBLEU

One limitation of BLEU is that it requires pre-tokenized text. To solve this issue, SacreBLEU is used, which standardizes the tokenization process.

Installing SacreBLEU

!pip install sacrebleu

Loading SacreBLEU

import evaluate
metric = evaluate.load("sacrebleu")

Rules for Using SacreBLEU

Predictions: List of strings
References: List of list of strings
It supports multiple acceptable translations.

Example

predictions = [
"This plugin lets you translate web pages between several languages automatically."
]
references = [[
"This plugin allows you to automatically translate web pages between several languages."
]]
metric.compute(predictions=predictions, references=references)
In this example, the BLEU score is approximately 46.75, which is a good result.
Comparatively, the Transformer model in the “Attention Is All You Need” research paper achieved a BLEU score of 41.8 for English-to-French translation.

Examples of Poor Translations

Excessive Repetition:
predictions = ["This This This This"]
In this case, the BLEU score is very low.
Excessively Short Sentences:
predictions = ["This plugin"]
In this case, the score is nearly zero.

BLEU Score Range

0 to 100 — Higher is better.

Compute Metrics Function

The following function converts model outputs into text and calculates the BLEU score.

```
import numpy as np
def compute_metrics(eval_preds):
preds, labels = eval_preds
if isinstance(preds, tuple):
preds = preds[0]
decoded_preds = tokenizer.batch_decode(
preds, skip_special_tokens=True
)
labels = np.where(
labels != -100,
labels,
tokenizer.pad_token_id
)
decoded_labels = tokenizer.batch_decode(
labels, skip_special_tokens=True
)
decoded_preds = [pred.strip() for pred in decoded_preds]
decoded_labels = [[label.strip()] for label in decoded_labels]
result = metric.compute(
predictions=decoded_preds,
references=decoded_labels
)
return {"bleu": result["score"]}
```

Function Workflow

Step: Predictions Decode
Explanation: Converts token IDs into text.
Step: Labels Decode
Explanation: Converts the reference translations into text.
Step: Replace -100
Explanation: Ignores padding tokens during evaluation.
Step: Post-processing
Explanation: Removes unnecessary spaces.
Step: SacreBLEU Calculation
Explanation: Determines the quality of the translation.

Fine-Tuning the Model (Complete Training Pipeline)

Now we will fully fine-tune the translation model. In this stage, we will log in to Hugging Face Hub, define training arguments, train the model, evaluate it, and finally upload it to the Hub.

Logging into Hugging Face

To upload the model to the Model Hub, we first need to log in to Hugging Face.

In Notebook

```
from huggingface_hub import notebook_login
notebook_login()
```

This will open a widget where you can enter your Hugging Face account credentials.

In Terminal

```
huggingface-cli login
```

Defining Seq2SeqTrainingArguments

Next, we define the hyperparameters required for training. For Seq2SeqTrainer, we use Seq2SeqTrainingArguments.

```
from transformers import Seq2SeqTrainingArguments

args = Seq2SeqTrainingArguments(
"marian-finetuned-kde4-en-to-fr",
evaluation_strategy="no",
save_strategy="epoch",
learning_rate=2e-5,
per_device_train_batch_size=32,
per_device_eval_batch_size=64,
weight_decay=0.01,
save_total_limit=3,
num_train_epochs=3,
predict_with_generate=True,
fp16=True,
push_to_hub=True,
)
```

Important Parameters

Parameter | Explanation
learning_rate | Controls how fast the model learns
num_train_epochs | Total number of training cycles
per_device_train_batch_size | Batch size per device during training
per_device_eval_batch_size | Batch size during evaluation
weight_decay | Helps reduce overfitting
save_strategy="epoch" | Saves the model after each epoch
predict_with_generate=True | Uses generate() during evaluation
fp16=True | Enables faster training on modern GPUs
push_to_hub=True | Uploads model to Hugging Face Hub

By default, the repository will be created under your username.

Creating the Seq2SeqTrainer

Now we pass the model, dataset, and other components to the Trainer.

```
from transformers import Seq2SeqTrainer

trainer = Seq2SeqTrainer(
model=model,
args=args,
train_dataset=tokenized_datasets["train"],
eval_dataset=tokenized_datasets["validation"],
data_collator=data_collator,
tokenizer=tokenizer,
compute_metrics=compute_metrics,
)
```

Evaluation Before Training

Before starting fine-tuning, we check the initial performance of the model.

```
trainer.evaluate(max_length=max_length)
```

Example output:
{

'eval_loss': 1.6964,
'eval_bleu': 39.27,
'eval_runtime': 965.88
}

Explanation:
A BLEU score of 39 means the model is already capable of producing reasonably good translations.

Training the Model

trainer.train()

During training, the model is saved after each epoch.
If push_to_hub=True is enabled, the model is automatically uploaded to Hugging Face Hub.
This allows training to be resumed later on another machine if needed.

Evaluation After Training

trainer.evaluate(max_length=max_length)

Example output:
{
'eval_loss': 0.8558,
'eval_bleu': 52.94,
'epoch': 3.0
}

Here, the BLEU score improves by nearly 14 points, indicating a significant performance boost.

Uploading the Model to Hugging Face Hub

trainer.push_to_hub(
tags="translation",
commit_message="Training complete"
)

This command:
Uploads the latest version of the model

Automatically creates a Model Card
Adds metadata required for inference demos

Example repository link:
https://huggingface.co/username/marian-finetuned-kde4-en-to-fr

Custom Training Loop (Accelerate-based Full Training Pipeline)

Now we will walk through a complete custom training loop so you can easily modify it according to your needs. This is similar to previous chapters, but here we use **Hugging Face Accelerate**, which makes training faster and more efficient across GPUs/TPUs.

Preparing the Data for Training

First, we convert the dataset into PyTorch tensors and create DataLoaders.

```
from torch.utils.data import DataLoader

tokenized_datasets.set_format("torch")

train_dataloader = DataLoader(
    tokenized_datasets["train"],
    shuffle=True,
    collate_fn=data_collator,
    batch_size=8,
)

eval_dataloader = DataLoader(
    tokenized_datasets["validation"],
    collate_fn=data_collator,
    batch_size=8,
)
```

Explanation:

- `set_format("torch")` → converts dataset into PyTorch tensors
- `train_dataloader` → used for training

- `eval_dataloader` → used for evaluation
- `data_collator` → ensures dynamic padding

Reloading the Model

To start fine-tuning from scratch, we reload the pretrained model:

```
from transformers import AutoModelForSeq2SeqLM

model = AutoModelForSeq2SeqLM.from_pretrained(model_checkpoint)
```

Optimizer Setup

```
from torch.optim import AdamW

optimizer = AdamW(model.parameters(), lr=2e-5)
```

Purpose:

Updates model weights during training.

Using Accelerate

```
from accelerate import Accelerator

accelerator = Accelerator()

model, optimizer, train_dataloader, eval_dataloader = accelerator.prepare(
    model, optimizer, train_dataloader, eval_dataloader
)
```

Benefits:

- Works across CPU, GPU, TPU
- Supports multi-GPU training
- Optimizes performance automatically

Learning Rate Scheduler

```
from transformers import get_scheduler

num_train_epochs = 3
num_update_steps_per_epoch = len(train_dataloader)
num_training_steps = num_train_epochs * num_update_steps_per_epoch

lr_scheduler = get_scheduler(
    "linear",
    optimizer=optimizer,
    num_warmup_steps=0,
    num_training_steps=num_training_steps,
)
```

What it does:

Gradually decreases learning rate for stable training.

Creating Hugging Face Repository

```
from huggingface_hub import Repository, get_full_repo_name

model_name = "marian-finetuned-kde4-en-to-fr-accelerate"
repo_name = get_full_repo_name(model_name)

output_dir = "marian-finetuned-kde4-en-to-fr-accelerate"
repo = Repository(output_dir, clone_from=repo_name)
```

Purpose:

- Stores model locally
- Enables automatic push to Hugging Face Hub

Post-processing Function (for BLEU)

```
import numpy as np

def postprocess(predictions, labels):
```

```
    predictions = predictions.cpu().numpy()
    labels = labels.cpu().numpy()

    decoded_preds = tokenizer.batch_decode(
        predictions, skip_special_tokens=True
    )

    labels = np.where(labels != -100, labels, tokenizer.pad_token_id)
    decoded_labels = tokenizer.batch_decode(
        labels, skip_special_tokens=True
    )

    decoded_preds = [pred.strip() for pred in decoded_preds]
    decoded_labels = [[label.strip()] for label in decoded_labels]

    return decoded_preds, decoded_labels
```

Purpose:

Prepares text for BLEU score calculation.

Full Custom Training Loop

```
from tqdm.auto import tqdm
import torch

progress_bar = tqdm(range(num_training_steps))

for epoch in range(num_train_epochs):

    # =========================
    # Training Phase
    # =========================
    model.train()

    for batch in train_dataloader:
        outputs = model(**batch)
        loss = outputs.loss

        accelerator.backward(loss)

        optimizer.step()
        lr_scheduler.step()
```

```
    optimizer.zero_grad()

    progress_bar.update(1)

# ========================
# Evaluation Phase
# ========================
model.eval()

for batch in tqdm(eval_dataloader):
    with torch.no_grad():
        generated_tokens = accelerator.unwrap_model(model).generate(
            batch["input_ids"],
            attention_mask=batch["attention_mask"],
            max_length=128,
        )

    labels = batch["labels"]

    generated_tokens = accelerator.pad_across_processes(
        generated_tokens,
        dim=1,
        pad_index=tokenizer.pad_token_id,
    )

    labels = accelerator.pad_across_processes(
        labels,
        dim=1,
        pad_index=-100,
    )

    predictions_gathered = accelerator.gather(generated_tokens)
    labels_gathered = accelerator.gather(labels)

    decoded_preds, decoded_labels = postprocess(
        predictions_gathered, labels_gathered
    )

    metric.add_batch(
        predictions=decoded_preds,
        references=decoded_labels,
    )

results = metric.compute()
```

```
    print(f"epoch {epoch}, BLEU score: {results['score']:.2f}")

    # ========================
    # Save & Push to Hub
    # ========================
    accelerator.wait_for_everyone()

    unwrapped_model = accelerator.unwrap_model(model)
    unwrapped_model.save_pretrained(
        output_dir,
        save_function=accelerator.save,
    )

    if accelerator.is_main_process:
        tokenizer.save_pretrained(output_dir)
        repo.push_to_hub(
            commit_message=f"Training in progress epoch {epoch}",
            blocking=False,
        )
```

Example Output

```
epoch 0, BLEU score: 53.47
epoch 1, BLEU score: 54.24
epoch 2, BLEU score: 54.44
```

Interpretation:

Each epoch improves performance → model is learning effectively.

Using the Fine-Tuned Model

```
from transformers import pipeline

model_checkpoint = "huggingface-course/marian-finetuned-kde4-en-to-fr"
translator = pipeline("translation", model=model_checkpoint)

translator("Default to expanded threads")
```

Output:

Par défaut, développer les fils de discussion

Another example:

```
translator(
    "Unable to import %1 using the OFX importer plugin. This file is not the correct format."
)
```

Output:

Impossible d'importer %1 en utilisant le module externe d'importation OFX. Ce fichier n'est pas le bon format.

Summary

Step	Description
DataLoader	Prepares data for training & evaluation
AutoModelForSeq2SeqLM	Loads pretrained translation model
AdamW	Optimizer for weight updates
Accelerator	Enables fast multi-device training
Scheduler	Controls learning rate
Custom Loop	Full control over training process
BLEU Score	Measures translation quality
Hugging Face Hub	Stores and shares model